

Arquitectura Hexagonal

1. ¿Qué es la Arquitectura Hexagonal y para qué sirve?

La Arquitectura Hexagonal — también llamada **Arquitectura de Puertos y Adaptadores** — fue propuesta por Alistair Cockburn en 2005 como una solución al problema del acoplamiento entre la lógica de negocio y los detalles técnicos (bases de datos, APIs, interfaces de usuario). El nombre "hexagonal" no implica que haya exactamente seis lados, sino que representa la idea de un núcleo central (el dominio) con múltiples puertos a través de los cuales se comunica con el exterior, permitiendo conectar y desconectar adaptadores según se necesite.

¿Para qué sirve?

- Aislar la lógica de negocio del mundo exterior para que pueda evolucionar sin romperse, incluso cuando cambian las tecnologías que la rodean.
- Permitir probar el negocio sin necesidad de infraestructura real (sin base de datos, sin red), lo que acelera el ciclo de desarrollo y mejora la calidad.
- Poder cambiar tecnologías (SQLite a PostgreSQL, REST a GraphQL, CLI a Web) sin reescribir el núcleo, lo que reduce el riesgo de los cambios.
- Retrasar decisiones tecnológicas hasta el último momento responsable: primero se construye el dominio puro y luego se decide qué base de datos o framework usar.
- Facilitar la colaboración en paralelo de varios equipos o desarrolladores, cada uno trabajando en un adaptador distinto contra un contrato común (el puerto).

¿Qué es el acoplamiento fuerte y por qué es un problema?

El acoplamiento mide el grado de dependencia entre módulos. Un acoplamiento fuerte ocurre cuando un módulo conoce detalles internos de otro (por ejemplo, importa directamente una librería concreta o asume la estructura de una respuesta HTTP). Esto provoca que un cambio en un módulo de bajo nivel (como la base de datos o un servicio externo) obligue a modificar muchos otros módulos, incluso aquellos que contienen reglas de negocio. El acoplamiento fuerte vuelve el código frágil, difícil de probar y costoso de mantener. La arquitectura hexagonal elimina el acoplamiento fuerte haciendo que todas las dependencias apunten hacia abstracciones (puertos) y nunca hacia implementaciones concretas.

¿Cuándo usarla?

- Proyectos con lógica de negocio no trivial que probablemente vivirán más de unos meses y necesitarán evolucionar sin reescrituras masivas.
- Equipos que trabajan en paralelo (uno en UI, otro en persistencia, otro en lógica), donde los contratos permiten integración tardía sin fricciones.
- Aplicaciones donde se anticipan cambios en proveedores externos, bases de datos o frameworks de UI; la hexagonal absorbe esos cambios sin afectar al núcleo.
- Cuando necesitas tests unitarios ultrarrápidos que no dependan de conexiones reales y permitan integración continua fluida.
- Sistemas donde el dominio es complejo (muchas reglas, validaciones, flujos) y merece ser modelado con independencia de la infraestructura.

¿Cuándo NO usarla?

- Prototipos desechables de una tarde o scripts que se ejecutan una única vez.
- Aplicaciones con lógica trivial (un CRUD sin reglas de negocio) donde el costo de la abstracción supera los beneficios.
- Proyectos con un único desarrollador y ciclo de vida corto, donde la separación estricta puede ralentizar la entrega inicial.
- Situaciones donde los requisitos tecnológicos son completamente inamovibles y nunca se prevén cambios de base de datos, API o interfaz.

2. El problema de fondo: acoplamiento

En aplicaciones tradicionales sin arquitectura definida, es común ver este patrón donde la función mezcla peticiones HTTP, lógica de negocio, persistencia y presentación:

```
def mostrar_clima(ciudad):
    respuesta = requests.get(f"https://api.openweathermap.org/data/2.5/weather?
q={ciudad}&appid=KEY")
    datos = respuesta.json()
    temperatura = datos["main"]["temp"] - 273.15
    conn = sqlite3.connect("clima.db")
    conn.execute("INSERT INTO historico VALUES (?, ?, ?)", (ciudad,
temperatura, datetime.now()))
    st.write(f"La temperatura en {ciudad} es {temperatura:.1f}°C")
```

Este código es un ejemplo de acoplamiento fuerte:

- Depende directamente de la librería `requests` (si mañana se cambia por `httpx`, hay que modificar esta función).
- Conoce la URL y la estructura exacta del JSON de OpenWeather (si la API cambia, se rompe).
- Usa `sqlite3` de forma directa (cambiar a PostgreSQL obliga a reescribir la persistencia en medio de la lógica).
- La salida está acoplada a Streamlit (`st.write`), impidiendo reutilizar la función en una CLI o en una API REST.

Consecuencias:

- No puedes probar `mostrar_clima` sin internet ni sin base de datos.
- Si cambia la estructura del JSON de OpenWeather, tocas esta función.
- Si quieres cambiar a otra API, tienes que reescribir toda la función.
- Si quieres mostrar los datos en consola en lugar de Streamlit, no puedes.
- La responsabilidad está mezclada: llamada HTTP + lógica de negocio + persistencia + presentación.

La solución hexagonal: cada responsabilidad se separa en una capa distinta y las dependencias apuntan hacia adentro. El dominio expresa lo que necesita mediante puertos y los adaptadores implementan esos puertos con tecnologías concretas. Así el código que cambia por razones técnicas nunca se mezcla con el que cambia por razones de negocio.

3. Comparación con otras arquitecturas

Aspecto	Arquitectura Hexagonal	Arquitectura en Capas (tradicional)	MVC	Arquitectura Limpia (Clean Architecture)
Separación	Lógica de negocio aislada por puertos	Capas horizontales (presentación → negocio → datos)	Modelo-Vista-Controlador	Similar a hexagonal pero con más niveles
Dirección de dependencias	Hacia el núcleo (adentro)	Hacia abajo (generalmente)	El controlador conoce al modelo	Hacia el núcleo (reglas de negocio)
Facilidad de test	Alta: el núcleo se prueba con fakes sin infraestructura	Media: las capas superiores dependen de inferiores	Media: el modelo suele estar acoplado	Alta: misma filosofía que hexagonal
Cambio de tecnología	Mínimo impacto (solo adaptador)	Puede requerir cambios en varias capas	Depende de la implementación	Mínimo impacto
Complejidad inicial	Media-alta	Baja	Baja-media	Alta
Cuándo usarla	Apps con lógica de negocio rica y cambiante	Prototipos, CRUDs simples	Aplicaciones web con UI y datos	Sistemas grandes y críticos

Relación con Arquitectura Limpia: La hexagonal es conceptualmente muy similar a la Clean Architecture de Robert C. Martin. Ambas comparten el Principio de Inversión de Dependencias (DIP). La Clean Architecture añade más niveles de abstracción (entidades, casos de uso, adaptadores de interfaz, frameworks). Hexagonal es más simple y práctica para el día a día. También está fuertemente relacionada con el Domain-Driven Design (DDD), que aporta patrones tácticos para modelar el dominio con precisión.

4. Profundizando en cada capa

4.1 El Dominio (el hexágono)

El dominio es el corazón de la aplicación. Contiene dos tipos de elementos: entidades y puertos. Aquí no hay frameworks, ni llamadas de red, ni SQL. Solo el lenguaje del negocio.

4.1.1 Entidades (Modelos de dominio)

Son objetos que representan conceptos del negocio con sus reglas intrínsecas. No son simples contenedores de datos; encapsulan comportamiento y validaciones que deben ser verdaderas siempre, sin importar quién los use. Por ejemplo, una medición climática sabe cómo convertir a Kelvin y si la temperatura es extrema.

```

from dataclasses import dataclass
from datetime import datetime

@dataclass
class MedicionClima:
    ciudad: str
    temperatura_c: float
    humedad: int
    presion_hpa: int
    descripcion: str
    timestamp: datetime

    def a_kelvin(self) -> float:
        return self.temperatura_c + 273.15

    def es_extremo(self) -> bool:
        return self.temperatura_c > 40 or self.temperatura_c < -10

```

Lo que debes saber:

- Las entidades no deben depender de nada externo (ni frameworks, ni librerías de infraestructura). Si una entidad importa `requests` o `sqlite3`, está mal.
- Deben contener reglas que sean invariantes del negocio. Por ejemplo, si la temperatura en Kelvin siempre debe ser positiva, esa validación estaría aquí.
- Si la regla cambia, solo cambia la entidad. El resto del sistema no se entera.

4.1.2 Puertos (interfaces)

Los puertos son contratos que definen cómo el dominio interactúa con el exterior. Hay dos tipos:

Puertos primarios (driving ports): Definen la interfaz que la aplicación ofrece a los actores externos (UI, tests, CLI). Generalmente son interfaces que encapsulan los casos de uso. El dominio las implementa, y los adaptadores primarios las consumen. Esto garantiza que quien invoca al sistema (por ejemplo, una vista web) dependa de una abstracción y no de una implementación concreta.

Puertos secundarios (driven ports): Definen qué necesita el dominio del exterior (persistencia, APIs externas, servicios de notificación). Son interfaces que el dominio requiere y los adaptadores secundarios implementan. El dominio expresa sus necesidades en sus propios términos, sin conocer detalles tecnológicos.

La inversión de dependencias es clave: en lugar de que el dominio importe una clase concreta de base de datos, es la base de datos la que implementa el puerto definido por el dominio. Así se invierte la flecha de la dependencia.

```

from abc import ABC, abstractmethod
from dominio.entidades import MedicionClima

```

```

class ServicioClima(ABC):
    """Puerto secundario: contrato para obtener datos climáticos externos."""

    @abstractmethod
    def obtener_actual(self, ciudad: str) -> MedicionClima:
        raise NotImplementedError

class RepositorioClima(ABC):
    """Puerto secundario: contrato para persistir y recuperar datos."""

    @abstractmethod
    def guardar(self, medicion: MedicionClima) -> None:
        raise NotImplementedError

    @abstractmethod
    def obtener_historico(self, ciudad: str, limite: int = 30) ->
list[MedicionClima]:
        raise NotImplementedError

class ConsultaClima(ABC):
    """Puerto primario: caso de uso que la UI o los tests invocarán."""

    @abstractmethod
    def ejecutar(self, ciudad: str) -> MedicionClima:
        raise NotImplementedError

```

Regla de oro: Los puertos solo contienen firmas de métodos, nunca lógica. Son 100% abstractos. Si un puerto tiene un cuerpo de método o importa alguna librería externa, deja de ser un contrato puro.

4.1.3 Casos de uso (Use Cases)

Orquestan el flujo de la aplicación usando los puertos. Cada caso de uso implementa un puerto primario y utiliza los puertos secundarios que necesita. Reciben sus dependencias por constructor (inyección) y nunca las crean internamente. Esta es la aplicación directa del Principio de Inversión de Dependencias.

```

from dominio.puertos import ServicioClima, RepositorioClima, ConsultaClima
from dominio.entidades import MedicionClima

class ObtenerClimaActual(ConsultaClima):
    def __init__(self, servicio_clima: ServicioClima, repositorio:
RepositorioClima):
        self._servicio = servicio_clima
        self._repositorio = repositorio

    def ejecutar(self, ciudad: str) -> MedicionClima:
        try:
            medicion = self._servicio.obtener_actual(ciudad)
            self._repositorio.guardar(medicion)
            return medicion
        except Exception:

```

```
historico = self._repositorio.obtener_historico(ciudad, limite=1)
if not historico:
    raise RuntimeError(f"No hay datos disponibles para {ciudad}")
return historico[0]
```

Características clave:

- Dependen de puertos (abstracciones), no de implementaciones concretas.
- No importan nada de infraestructura (no `import requests`, no `import sqlite3`).
- Son el lugar donde va la lógica de negocio orquestada.
- Son fácilmente testeables con implementaciones falsas.
- Al implementar un puerto primario, permiten que los adaptadores primarios también dependan de una abstracción.

4.1.4 Manejo de transacciones y asincronía

En aplicaciones reales a menudo se necesita coordinar varias operaciones bajo una misma transacción o manejar operaciones de entrada/salida de forma asíncrona. La arquitectura hexagonal se adapta naturalmente:

- Para **transacciones**, se define un puerto secundario como `UnidadDeTrabajo` que agrupa repositorios y expone métodos `commit()` y `rollback()`. El caso de uso recibe la unidad de trabajo en lugar de repositorios sueltos, y la infraestructura real gestiona la transacción (por ejemplo, con un contexto de SQLAlchemy).
- Para **asincronía**, los puertos pueden declarar métodos `async`. Los adaptadores reales utilizan clientes asíncronos (`aiohttp`, `aiosqlite`), mientras que los fakes simulan el mismo comportamiento sin I/O real. Los casos de uso se implementan con `async def` y el punto de ensamblaje ejecuta el bucle de eventos adecuado.

4.2 Los Adaptadores

Son las implementaciones concretas de los puertos. Viven fuera del hexágono. Su responsabilidad es traducir entre el mundo exterior (formatos de red, SQL, HTML) y el lenguaje del dominio.

4.2.1 Adaptadores secundarios (driven adapters)

Conectan el dominio con el mundo exterior: APIs, bases de datos, sistemas de archivos.

Buenas prácticas:

- Cada adaptador implementa exactamente un puerto.
- El adaptador se encarga de la traducción entre el formato externo y el del dominio. Por ejemplo, convierte un JSON de respuesta en una instancia de `MedicionClima`.
- Todo el código específico de tecnología (SQL, HTTP, JSON) vive aquí.

```
# adaptadores/api/openweather_adapter.py
import requests
```

```

import datetime
from dominio.puertos import ServicioClima
from dominio.entidades import MedicionClima

class OpenWeatherAdapter(ServicioClima):
    def __init__(self, api_key: str):
        self._api_key = api_key
        self._base_url = "https://api.openweathermap.org/data/2.5/weather"

    def obtener_actual(self, ciudad: str) -> MedicionClima:
        response = requests.get(
            self._base_url,
            params={"q": ciudad, "appid": self._api_key, "units": "metric"}
        )
        response.raise_for_status()
        datos = response.json()
        return MedicionClima(
            ciudad=ciudad,
            temperatura_c=datos["main"]["temp"],
            humedad=datos["main"]["humidity"],
            presion_hpa=datos["main"]["pressure"],
            descripcion=datos["weather"][0]["description"],
            timestamp=datetime.datetime.now()
        )

```

4.2.2 Adaptadores primarios (driving adapters)

Son los puntos de entrada que el usuario usa para interactuar con la aplicación: interfaces web, CLIs, APIs REST, tests. Dependen de un puerto primario, no de la implementación concreta del caso de uso. Esto permite que la misma interfaz gráfica pueda trabajar con un caso de uso real o con uno de prueba.

```

# adaptadores/presentacion/streamlit_ui.py
import streamlit as st
from dominio.puertos import ConsultaClima

def mostrar_clima(caso_uso: ConsultaClima):
    st.title("Clima Actual")
    ciudad = st.text_input("Ciudad", "Madrid")
    if st.button("Consultar"):
        with st.spinner("Consultando..."):
            try:
                medicion = caso_uso.ejecutar(ciudad)
                st.metric("Temperatura", f"{medicion.temperatura_c:.1f}°C")
                st.metric("Humedad", f"{medicion.humedad}%")
            except Exception as e:
                st.error(f"Error: {e}")

```

4.2.3 Adaptadores falsos (Fakes) para testing

Los fakes son implementaciones ligeras y funcionales de un puerto. A diferencia de los mocks, que solo verifican llamadas, los fakes tienen un comportamiento real (por ejemplo, devuelven datos predefinidos). Esto los hace más mantenibles y menos frágiles ante cambios en la implementación interna.

```
# adaptadores/api/fake_servicio.py
from dominio.puertos import ServicioClima
from dominio.entidades import MedicionClima
from datetime import datetime

class FakeServicioClima(ServicioClima):
    def __init__(self, temperatura_fija: float = 25.0):
        self._temperatura = temperatura_fija

    def obtener_actual(self, ciudad: str) -> MedicionClima:
        return MedicionClima(
            ciudad=ciudad,
            temperatura_c=self._temperatura,
            humedad=60,
            presion_hpa=1013,
            descripcion="Soleado (simulado)",
            timestamp=datetime.now()
        )
```

4.3 El Punto de Ensamblaje (Composition Root)

Es el único lugar del código donde se crean instancias concretas y se conectan adaptadores a casos de uso. Es el "pegamento" de la aplicación. Centralizar la composición de objetos evita que las dependencias concretas se dispersen por el código y permite cambiar la configuración de la aplicación con solo modificar este punto.

```
# app_streamlit.py (punto de ensamblaje)
import os
from adaptadores.api.openweather_adapter import OpenWeatherAdapter
from adaptadores.persistencia.sqlite_adapter import SQLiteAdapter
from adaptadores.presentacion.streamlit_ui import mostrar_clima
from dominio.casos_de_uso import ObtenerClimaActual

def main():
    api_key = os.getenv("OPENWEATHER_API_KEY")
    servicio = OpenWeatherAdapter(api_key)
    repositorio = SQLiteAdapter("clima.db")
    caso_uso = ObtenerClimaActual(servicio, repositorio)
    mostrar_clima(caso_uso)

if __name__ == "__main__":
    main()
```


Alternativa para pruebas:

```
# tests/test_obtener_clima_actual.py
from dominio.casos_de_uso import ObtenerClimaActual
from adaptadores.api.fake_servicio import FakeServicioClima
from adaptadores.persistencia.fake_repositorio import FakeRepositorioClima

def test_obtener_clima_actual():
    servicio = FakeServicioClima(temperatura_fija=30.0)
    repositorio = FakeRepositorioClima()
    caso_uso = ObtenerClimaActual(servicio, repositorio)

    resultado = caso_uso.ejecutar("Madrid")

    assert resultado.temperatura_c == 30.0
    assert resultado.ciudad == "Madrid"
    assert repositorio.ultimo_guardado is not None
```

Ventaja clave: El mismo caso de uso se prueba con fakes (test unitario) y en producción con adaptadores reales. No cambia ni una línea del dominio.

5. Ejemplos de uso y malas prácticas

5.1 Buenas prácticas

Práctica	Explicación
Un puerto = una responsabilidad	No crees un puerto gigante con métodos de API y persistencia mezclados. Cada puerto debe tener una única razón de cambio.
Dependencias en constructor	Los casos de uso reciben sus dependencias por constructor (inyección), nunca las crean internamente.
El dominio usa lenguaje de negocio	<code>obtener_actual(ciudad)</code> en lugar de <code>fetch_data_from_url(url, params)</code> . El código debe leerse como una narración del proceso de negocio.
Adaptadores traducen formatos	El adaptador convierte el JSON de la API a <code>MedicionClima</code> . El dominio nunca ve el JSON crudo.
Un adaptador por tecnología	Si necesitas OpenWeather + WeatherAPI, son dos adaptadores separados que implementan el mismo puerto.
Tests con fakes, no con mocks pesados	Los fakes son implementaciones funcionales ligeras. Más mantenibles que mocks excesivos porque no necesitan configurar expectativas detalladas.
El punto de ensamblaje es el único lugar concreto	Solo ahí se sabe qué adaptador concreto se usa. El resto del código depende de abstracciones.

Práctica	Explicación
Los adaptadores primarios dependen de puertos primarios	Igual que con los secundarios, se debe depender de una abstracción, no de la clase concreta del caso de uso.

5.2 Malas prácticas

Práctica	Problema	Cómo evitarlo
Poner lógica en los puertos	Los puertos dejan de ser contratos puros y mezclan responsabilidades.	Los puertos solo tienen métodos abstractos. Cero implementación.
El caso de uso crea sus propios adaptadores	<code>self._servicio = OpenWeatherAdapter("key")</code> dentro del caso de uso acopla el núcleo a la tecnología.	Inyecta siempre por constructor.
Puertos que cambian cuando cambia el adaptador	Si OpenWeather requiere un método nuevo, el puerto no debería modificarse.	El puerto refleja necesidades del dominio, no capacidades del adaptador.
Adaptadores que lanzan excepciones técnicas	<code>sqlite3.OperationalError</code> llega al caso de uso, que no sabe manejarla.	Los adaptadores envuelven excepciones técnicas en excepciones del dominio.
El dominio importa librerías externas	<code>import pandas</code> dentro de un caso de uso acopla el núcleo.	Si necesitas pandas, úsalo en el adaptador o crea un servicio separado.
Demasiados puertos pequeños	Micro-puertos con un solo método que generan sobreingeniería.	Agrupar operaciones relacionadas. Un repositorio con 5 métodos está bien.
Punto de ensamblaje disperso	Instancias creadas en varios archivos, difícil de rastrear.	Un único archivo <code>main.py</code> , <code>app.py</code> o <code>container.py</code> .
Anular la inyección con variables globales	Usar <code>settings.API_KEY</code> directamente en el adaptador.	Pasa configuración por constructor.

Práctica	Problema	Cómo evitarlo
Adaptador primario acoplado a la implementación del caso de uso	Al tipar directamente <code>ObtenerClimaActual</code> en lugar del puerto <code>ConsultaClima</code> se pierde la posibilidad de sustituir el caso de uso en pruebas o con otros adaptadores.	El adaptador primario siempre debe recibir el puerto (interfaz) correspondiente.

5.3 Ejemplo de mala práctica y su corrección

Incorrecto:

```
# casos_de_uso/obtener_clima_actual.py
import requests
import sqlite3
import streamlit as st

def ejecutar(ciudad):
    resp = requests.get(f"https://api.openweathermap.org/data/2.5/weather?q={ciudad}&appid=KEY")
    data = resp.json()
    temp = data["main"]["temp"] - 273.15
    conn = sqlite3.connect("clima.db")
    conn.execute("INSERT INTO historico VALUES (?, ?)", (ciudad, temp))
    st.metric("Temperatura", f"{temp:.1f}°C")
```

Correcto:

```
# casos_de_uso/obtener_clima_actual.py
from dominio.puertos import ServicioClima, RepositorioClima, ConsultaClima
from dominio.entidades import MedicionClima

class ObtenerClimaActual(ConsultaClima):
    def __init__(self, servicio: ServicioClima, repositorio: RepositorioClima):
        self._servicio = servicio
        self._repositorio = repositorio

    def ejecutar(self, ciudad: str) -> MedicionClima:
        medicion = self._servicio.obtener_actual(ciudad)
        self._repositorio.guardar(medicion)
        return medicion
```

6. Experiencias de trabajo con Arquitectura Hexagonal

Experiencia 1: El cambio de base de datos que no dolió

Trabajábamos en una aplicación que guardaba métricas en SQLite durante desarrollo. Al llegar a producción, el equipo de infraestructura exigía PostgreSQL. Como el repositorio era un puerto, solo necesitamos escribir `PostgresAdapter` que implementara `RepositorioClima`. El cambio en el punto de ensamblaje fue de una línea. El domino completo (entidades, casos de uso, reglas de negocio) no se tocó. Los tests unitarios ya validaban que la lógica funcionaba con fakes; los tests de integración en PostgreSQL confirmaban que el adaptador funcionaba. El despliegue fue en el sprint previsto.

Experiencia 2: Cuando el equipo trabaja en paralelo

Tres desarrolladores trabajando simultáneamente: uno en la UI con Streamlit, otro en la integración con OpenWeather, otro en la lógica de estadísticas. Los puertos se definieron el primer día. Cada uno trabajaba contra los puertos con fakes. Cuando llegó el momento de integrar, todo encajó sin conflictos. Sin la separación hexagonal, habría sido imposible probar la UI sin la API real, o probar estadísticas sin la base de datos.

Experiencia 3: El exceso de ingeniería (antipatrón)

Un equipo aplicó hexagonal a un microservicio que era básicamente un CRUD con dos tablas. Terminaron con 7 puertos, 12 clases y 4 adaptadores para lo que se resolvía con 3 archivos. La lección: la arquitectura hexagonal es una herramienta, no un dogma. Para lógica simple, un diseño en capas ligero es más apropiado.

Experiencia 4: El falso sentido de seguridad

"Tenemos arquitectura hexagonal, así que nuestra lógica está aislada". Pero en la práctica, los casos de uso tenían acceso a la base de datos directamente porque alguien importó `RepositorioClima` como concreto en lugar de abstracto. La arquitectura se sostiene con disciplina: si alguien rompe la regla de dependencia, el beneficio desaparece. Las revisiones de código y herramientas como `lint` con reglas de importación ayudan a mantener la integridad.

7. Reglas prácticas para aplicar hexagonal sin morir en el intento

1. **Empieza pequeño:** Un puerto, un caso de uso, dos adaptadores. Crece cuando lo necesites. No diseñes docenas de puertos desde el día uno.
2. **Define los puertos primero:** Antes de escribir cualquier implementación, define las interfaces que necesita tu dominio. Esto obliga a pensar en las necesidades reales del negocio.
3. **El dominio nunca sabe qué hay fuera:** Si un caso de uso necesita `import requests`, algo está mal. Esa es la prueba del algodón.
4. **No uses herencia en los adaptadores a menos que realmente compartan lógica.** La composición es mejor y evita jerarquías rígidas.
5. **Mantén los fakes simples:** Un `FakeRepositorio` con un `list` en memoria es suficiente. No necesita simular todos los casos extremos; solo lo necesario para el test.
6. **Un solo punto de ensamblaje:** Toda construcción de dependencias en un solo lugar. Usa un contenedor simple si es necesario.
7. **No mezcles lógica de presentación con lógica de negocio:** Streamlit no debe aparecer en el dominio. La vista es solo un traductor de eventos y datos.

8. **Usa tipos (type hints):** Ayudan a que los contratos sean explícitos y las herramientas de verificación los validen.
 9. **Mantén la simetría:** Los adaptadores primarios también deben depender de un puerto primario (interfaz), no de la implementación concreta.
 10. **Considera la asincronía y las transacciones desde el inicio si son relevantes.** Define puertos con `async` cuando corresponda y modela la unidad de trabajo como un puerto secundario.
-

8. Autoevaluación

Preguntas conceptuales

1. ¿Cuál es la principal diferencia entre un puerto primario y uno secundario? Da un ejemplo de cada uno.
2. Explica con tus palabras el Principio de Inversión de Dependencias (DIP) y cómo lo aplica la arquitectura hexagonal.
3. ¿Qué problema resuelve el punto de ensamblaje (Composition Root)? ¿Por qué no deberían estar las instancias dispersas por el código?
4. ¿Cuándo NO recomendarías usar arquitectura hexagonal? Menciona al menos tres escenarios.
5. ¿Cuál es la diferencia entre un test unitario y un test de integración en el contexto de una aplicación hexagonal?
6. ¿Por qué el dominio no debe importar librerías como `requests`, `sqlite3` o `pandas`? ¿Dónde deberían usarse?
7. Compara arquitectura hexagonal con arquitectura en capas tradicional. ¿Qué ventajas y desventajas tiene cada una?
8. ¿Qué es un "adaptador falso" (fake) y por qué es preferible a usar mocks excesivos en muchos casos?
9. ¿Qué le pasa al diseño si un caso de uso crea sus propias dependencias con `new` o el constructor directamente?
10. ¿Cómo afecta la arquitectura hexagonal al despliegue continuo y a la integración continua?

Ejercicios prácticos

Ejercicio 1: Identificar violaciones

Dado este código, identifica al menos 5 violaciones de los principios de arquitectura hexagonal y propón cómo corregirlas:

```
import requests
import json
import sqlite3
```

```
def procesar_clima(ciudad):
    api_key = "12345"
    url = f"http://api.openweathermap.org/data/2.5/weather?q={ciudad}&appid={api_key}"
    response = requests.get(url)
    data = response.json()

    temperatura = data["main"]["temp"] - 273.15
    humedad = data["main"]["humidity"]

    conn = sqlite3.connect("db.sqlite")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE IF NOT EXISTS clima (ciudad TEXT, temp REAL, humedad INT)")
    cursor.execute("INSERT INTO clima VALUES (?, ?, ?)", (ciudad, temperatura, humedad))
    conn.commit()
    conn.close()

    if temperatura > 30:
        print(f"Hace calor en {ciudad}")
    else:
        print(f"Temperatura agradable en {ciudad}")

    return {"temp": temperatura, "humedad": humedad}
```

Ejercicio 2: Diseñar una arquitectura hexagonal

Diseña la estructura de carpetas, puertos, casos de uso y adaptadores para un sistema de **gestión de pedidos de una tienda online** con los siguientes requisitos:

- Los clientes pueden crear pedidos con uno o más productos.
- El sistema debe verificar el stock contra un sistema externo de inventario.
- Los pedidos se guardan en una base de datos relacional.
- Cuando un pedido se confirma, se envía un email de notificación (servicio externo).
- Se puede consultar el histórico de pedidos de un cliente.

Define:

1. Las entidades del dominio.
2. Los puertos (interfaces) necesarios.
3. Los casos de uso.
4. Los adaptadores (mínimo 3).
5. El punto de ensamblaje (pseudocódigo).

Ejercicio 3: Refactorizar a hexagonal

Toma el código del Ejercicio 1 y refactorízalo siguiendo la arquitectura hexagonal. Escribe:

1. El puerto **ServicioClima** (interfaz).

2. El puerto `RepositorioClima` (interfaz).
3. El puerto primario `ConsultaClima` (interfaz del caso de uso).
4. El caso de uso `ProcesarClima` (implementa `ConsultaClima`).
5. El adaptador `OpenWeatherAdapter`.
6. El adaptador `SQLiteAdapter`.
7. Un `FakeServicioClima` para pruebas.
8. Un test unitario para `ProcesarClima` usando fakes.

Ejercicio 4: Cambio de tecnología

Partiendo de tu solución del Ejercicio 3, explica paso a paso qué tendrías que cambiar si:

- a) Quieres reemplazar SQLite por MongoDB.
- b) Quieres reemplazar OpenWeather por WeatherAPI.
- c) Quieres añadir una interfaz de línea de comandos (CLI) además de Streamlit.
- d) Quieres que el sistema funcione sin conexión usando un archivo JSON como fuente de datos.

Ejercicio 5: Análisis crítico

Lee estos escenarios y decide si aplicar arquitectura hexagonal es apropiado. Justifica tu respuesta:

- a) Un script de 50 líneas que descarga un archivo CSV, lo procesa con pandas y genera un gráfico.
- b) Una API REST con 3 endpoints que consultan una base de datos y devuelven JSON. El equipo es de 2 personas y el proyecto durará 6 meses.
- c) Un sistema bancario con reglas de negocio complejas (tasas de interés, cálculos de riesgo, validaciones de transacciones, integración con 5 proveedores externos). El equipo es de 12 personas y el proyecto durará 3 años.
- d) Una aplicación móvil de notas personales. Un desarrollador. Proyecto personal.

Solucionario (respuestas breves)

Pregunta 1: Un puerto primario define la interfaz que la aplicación ofrece a los actores externos (UI, tests) para que puedan invocar sus funcionalidades. Un puerto secundario define la interfaz que la aplicación necesita de proveedores externos (BD, APIs). El primario es un punto de entrada (ej: `ConsultaClima`), el secundario es un punto de salida (ej: `RepositorioClima`).

Pregunta 2: DIP dice que los módulos de alto nivel (dominio) no deben depender de módulos de bajo nivel (infraestructura). Ambos deben depender de abstracciones. Hexagonal lo aplica haciendo que el dominio dependa de puertos (abstracciones) y los adaptadores implementen esos puertos. Tanto el lado primario como el secundario se apoyan en esta inversión.

Pregunta 3: El punto de ensamblaje centraliza la creación de dependencias. Si las instancias se crean dispersas, pierdes la capacidad de cambiar implementaciones globalmente y el acoplamiento se filtra.

Pregunta 4: Prototipos desechables, scripts de una sola función, CRUDs sin lógica de negocio, proyectos de un fin de semana.

Pregunta 5: Test unitario prueba el caso de uso con adaptadores falsos (sin red, sin BD). Test de integración prueba el adaptador real contra su tecnología (ej: OpenWeatherAdapter llama realmente a la API).

Pregunta 6: El dominio debe ser puramente lógica de negocio, independiente de tecnologías. Si importa requests, no puedes probarlo sin red ni cambiar la librería HTTP sin tocar el núcleo. Esas librerías van en los adaptadores.

Pregunta 7: Capas tradicional: más simple al inicio, pero las capas inferiores (BD) contaminan las superiores. Hexagonal: más estructura inicial, pero mucho más fácil de testear y cambiar tecnologías.

Pregunta 8: Un fake es una implementación funcional ligera (ej: repositorio con lista en memoria). Es más mantenible que mocks porque tiene comportamiento real y no requiere configurar expectativas para cada test.

Pregunta 9: Acopla el caso de uso a una implementación concreta. No puedes cambiarla sin modificar el caso de uso. Rompe la inyección de dependencias y el Principio de Inversión.

Pregunta 10: Hexagonal facilita CI/CD porque puedes ejecutar tests unitarios rápidos sin infraestructura en cada commit, y reservar tests de integración para etapas posteriores. Los cambios en infraestructura no afectan la lógica de negocio.

9. Referencias y lecturas complementarias

- Cockburn, A. (2005). "Hexagonal Architecture" — alistair.cockburn.us/hexagonal-architecture
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.
- Fowler, M. (2004). "Inversion of Control Containers and the Dependency Injection pattern" — martinfowler.com